Supabase branching → highest-leverage Yitzchak work

Date: 2026-05-06 **Context:** Yitzchak is asking how to access L3 data and proposing he build the dashboard UI. Started as "how do I branch supabase" and ended at "his comparative advantage isn't UI."

Latest WhatsApp with Yitzchak (vita `l1_events`, channel `wa-972552631180`)

Today (May 6) — the active thread:

- 05:29 Yitzchak: *"if I want to access all the L3 data — final processed output grouped by source (WhatsApp, email, Xero, etc.) — where would I find it in Supabase?"*
- 08:01 Yitzchak: *"do you have a plan for the L3 structure? I'm trying to make the GENUI... I need date sources ie whatsapp, zero etc and then the content itself"*
- 08:48 Yitzchak: *"could we meet for a few minutes?"*
- 08:55 Mordechai: *"cant meet now rather send msg"*
- 08:57 Yitzchak: **"I'm building the dashboard UI... should I focus only on UI and come back to you to connect the data (like calling your MCP), or is there a better approach?"**

May 3 thread was the Zoom + wireframe handoff (`claude.ai/design/p/019dea9b…`). Before that, the chat is Xero OAuth coordination from late March / mid-April.

Turn 1 — How to technically branch Supabase

The vita project (`dkccadwohifcqcdzhhnu`) currently has one branch: `main`, status `MIGRATIONS_FAILED` — flag this before doing anything else.

Four real options, ranked by isolation:

	What it is	Cost	Data carried?	Good for
1. Supabase Preview Branch	`mcp__supabase__create_branch` or push a git branch with `supabase/config.toml` wired to GitHub. Spins up a fresh project_ref, runs all migrations from `supabase/migrations/`, gives Yitzchak his own URL + keys.	~$0.32/day per branch (Pro plan req'd)	❌ schema only — empty tables (`with_data: true` is beta)	Schema experiments, PR previews
2. Separate Supabase project	New free-tier project, run the same migrations, seed manually	Free	❌	Yitzchak full sandbox, fully isolated
3. Local Supabase	`supabase start` (Docker: Postgres + GoTrue + Storage + Studio) in vita repo	Free	seed via `supabase/seed.sql`	UI dev with hot reload, offline
4. Shared project +	Give him a read-only PostgREST role + limited anon key on prod	Free	✅ live data	Dashboard reading real

	What it is	Cost	Data carried?	Good for
RLS / role				L1/L2/L3

Mechanically for **(1)**:

```
# either via MCP
create_branch(project_id="dkccadwohifcqcdzhhnu", name="yitzchak-ui")
# or via GitHub integration: enable Branching in Supabase dashboard → push branch
```

Branch inherits migrations from `infra/supabase/migrations/` (the `.draft/` dir uncommitted — those won't ship until promoted). Production data does *not* copy.

Is branching the right approach here?

Probably not for what Yitzchak is actually asking. His question is *"where's the data and what's the contract"*, not *"give me a sandbox to mutate"*. Branching would hand him an empty DB and a ~$10/mo bill without answering either question.

What he actually needs, in order:

1. **A stable read contract** — the `l1_events` schema is the contract. L2/L3 are empty (`l2_syntheses=0`, `l3_surfaces=0`), so there's no "L3 grouped by source" to point him at *yet*. L3 doesn't exist as a table; it's a planned synthesis layer. For now `l1_events.channel_id → channels.kind` *is* the "grouped by source" cut.
2. **A read-only key + a 5-line query example** — anon key + an RLS policy that gates by his tenant, so his UI calls supabase-js directly. No MCP needed for read.
3. **A local supabase for his UI loop** — `supabase start` + a `seed.sql` snapshot of ~100 `l1_events` rows.
4. **Branch only when he writes migrations** — when he needs to add `l3_surfaces` columns or RPC functions. Then a preview branch is the right tool.

Fix `MIGRATIONS_FAILED` on main first — Yitzchak forking off a broken branch will compound the problem.

Turn 2 — Mordechai: "what's actually the highest-leverage UI stuff for Yitzchak? or other stuff that doesn't need to be UI"

First-pass answer (before reading the codebase carefully):

Vita data flow status:

- L0: 1,756 artifacts ✅
- L1: 102 events ✅
- L2 syntheses: **0** ❌
- L3 surfaces: **0** ❌
- Eval fixtures/runs: **0** ❌
- main branch: `MIGRATIONS_FAILED`

Yitzchak's comparative advantage (from his March work + chat history):

- Built steve-agent-2.0 end-to-end (OAuth, Xero sync, MongoDB, embeddings)
- Backend integrations and data plumbing

- Per `viter-workspace/CLAUDE.md` he's the named "infrastructure/audit role (Supabase buckets, security hardening)"

He's a **backend/integrations person doing UI work because there's a wireframe in his hand.** Tool-finds-job, not job-finds-tool.

First-pass leverage ranking

Tier 1 — only-Yitzchak-can-do-this: Xero L0→L1 extractor · Gmail extractor · RLS + tenant isolation audit · per-tenant storage buckets

Tier 2 — backend-of-the-dashboard: Postgres views/RPC for "L3 grouped by source" · eval harness wiring · fix `MIGRATIONS_FAILED`

Tier 3 — UI work (lowest leverage): the dashboard React itself — Bolt/v0/Lovable can crank it out against a stable view contract

---

Turn 3 — Mordechai: "filter thru what i built in viter also think clearly"

Course correction. Stop guessing — read the actual codebase. Here's what's really there.

What's actually built in vita

The data spine is mostly done:

- 19 supabase migrations (May 4–5), full L0/L1/L2/L3 schema with trigger-enforced citation integrity + staleness propagation
- 12 active extractors: claude-code jsonl, whatsapp zip parser, gowa webhook (live), whisper turbo (voice + meeting), gemini image-caption + PDF chunking, mammoth, sheetjs, html-strip, markdown, json
- 5 candidate meeting extractors benchmarked (assemblyai, grok, elevenlabs, pyannote, claude-diarizer) — current branch
- 1,756 L0 artifacts, 102 L1 events flowing
- Synthesizer package fully scaffolded (`citation-parser` , `llm.ts` , `prompts/` , `scopers/` , `rubric` w/ tests, `synthesizer.ts`)
- Adapters: gowa webhook (working), desktop/browser/file-watcher/hil/mcp (scaffolds)

The web app is further along than expected:

- `/chat` — **887 LOC**, real multi-channel inbox: `ChannelRail` , `ConversationView` , `MessageBubble` , `RealtimeStream` , `Composer` . **This *is* a dashboard.**
- `/spaces/[slug]/inbox` — early
- `/meetings` — uploader (current branch)
- `/settings/whatsapp` — device pairing
- `/api/media/[artifactId]` — signed URL serving
- Auth flow done

The actual gaps:

- `l2_syntheses = 0` — synthesizer code exists, **has never run on real data**
- `l3_surfaces = 0` — same
- `eval_runs = 0` — extractors flying blind
- No Xero adapter (you had one in steve-agent — Yitzchak built it)

- No Gmail adapter (channel scaffold exists, zero events)
- No calendar / screenpipe adapter
- `MIGRATIONS_FAILED` on supabase main

Yitzchak's "build the dashboard UI" is wrong on three counts

1. **The chat UI is already the multi-channel dashboard.** Building a parallel "GenUI dashboard" duplicates 887 LOC of working code.
2. **L2/L3 are empty** — there's no structured per-source data to render. He's asking "where's L3" because L3 *isn't there yet*. Building UI on top of empty tables is pixel-pushing.
3. **It's the wrong fit for him.** He shipped steve-agent (Xero OAuth + sync + Mongo + embeddings). He's a *backend-integrations* engineer. The dashboard wireframe is the only reason UI feels like the next move.

Refined leverage ranking (post-codebase-read)

Tier 1 — only-him, directly reuses steve-agent muscle:

1. **Port the Xero adapter from steve-agent → vita.** New `xero_connection` source_type, OAuth flow (he wrote the doc on this in WhatsApp Mar 30), per-tenant token vault, periodic sync, L0 = raw Xero JSON, L1 extractor = structured invoice/contact events. **Insperanto is a CFO firm — Xero data is what they actually pay for.** Without this, the Persofi pilot has no financial substrate.
2. **Gmail adapter, same template.** `mordechai-inbox` channel sits empty. OAuth + Gmail API watch + .eml → L0 → thread parser → L1. Second-highest-volume corpus after WhatsApp.

Tier 2 — unblocks everything downstream including his own UI question:

3. **Run `synthesize.ts` end-to-end for the first time.** The synthesizer is *built*. Somebody has to babysit it through its first 10 runs, fix the citation-trigger errors, validate output. Once `l2_syntheses` has rows, "where's L3" changes from "doesn't exist" to "here's a query."
4. **Wire up `eval_fixtures` + `eval_runs`.** Pick 5 fixtures per extractor, write the regression harness. With 12 active extractors and 5 candidates churning, this is operational debt that compounds.

Tier 3 — only after L2 has data:

5. **`/spaces/[slug]` proper** — a per-channel/per-source rendering of L2 syntheses. Not a parallel dashboard, an *additional surface* on top of `/chat`. Where his wireframe could actually land.
6. Fix `MIGRATIONS_FAILED` on main.

What to actually tell him

> *"Pause the dashboard. The chat UI at `apps/web/app/chat` is already the multi-channel inbox — 887 LOC, working realtime, all 14 channels. What's missing isn't UI, it's:*
>
> *(a) **Xero adapter** — port your steve-agent OAuth + sync into `vita/adapters/xero` and write the L0→L1 extractor. Insperanto is a CFO firm, this is the substrate they pay for, and you've already built it once.*
>
> *(b) **First L2 run.** `packages/runtime/src/synthesizers` is built but has never run on real data — `l2_syntheses` is 0. Babysit it through the first 10 runs, fix the citation-trigger errors, validate output.*
>
> *Once those exist, the GenUI dashboard becomes a 1-shot v0/Lovable prompt against the L2 contract. Right now you'd be hand-building React over empty tables."*

Compressed principle (intermediate)

> *A wireframe makes UI feel like the obvious work. But the UI is already built. What hasn't been built is the data layer he actually has the skills to build — and if he doesn't build it, you will, slowly, while juggling the meeting extractor.*

Turn 4 — Mordechai: "look at edge functions in viterAi/viter and viter db too"

Course correction #2. I'd been treating vita as the only world. viter is a *live production system* with paying users.

What's actually in viter

9 active edge functions, 2,313 LOC — Xero stack already built:

Function	LOC	Version	Status
`xero-callback`	116	v11	OAuth handler
`xero-connect`	36	v9	Initiates OAuth
`xero-sync`	**387**	v9	**Daily — last 2026-05-06 03:00 UTC, 8 syncs/7d**
`extract-statement`	**601**	v16	PDF/CSV → statement_lines (most code, gnarliest)
`process-statement`	214	v9	Post-processing
`run-reconciliation`	508	**v32**	Most-iterated; last 2026-05-05
`tag-priority`	50	v6	Match prioritization
`canary-check`	356	v3	**Hourly, 168 runs/7d**
`run-sql`	45	—	Privileged SQL

Live operational signal (viter DB, project `mcghcqbjtwtkdyezcswr`):

- xero-sync runs **daily** ✅
- canary-check runs **hourly** ✅ (168 runs / 7d)
- 16 statements uploaded last 7d — Insperanto actively using it
- 12,649 Xero invoices · 19,706 payments · 15 active OAuth tokens (tenant connections)
- 4,692 reconciliation matches across 147 runs
- 134,859 audit_log rows
- **45 views** (`v_*`) — bulk of dashboard logic in SQL
- `activity_log` **dead since 2026-04-19** ⚠️ — MCP feed stopped silently
- L0/L1/L2/L3 schema also started here, also empty (0 L1/L2/L3, but 9,793 chat turns + 83 L0)
- `mcp_tools_registry`, `mcp_pinned`, `mcp_findings` all empty — `/me` dashboard scaffolded but unfed
- `lib/tools/viter-tools.ts` = **1,193 LOC** of MCP tool definitions (no registry rows)

This rewrites my earlier analysis

Two corrections:

1. Earlier I said "no Xero adapter exists." **False — it's a 387-LOC edge function in viter, running daily.** vita doesn't have one; viter does.
2. Earlier I framed the bottleneck as "build new adapters." Bigger reality: **viter is a live production system with operational debt and a dead MCP feed**, while vita is the new platform. The work is *both* "operate viter" AND "bridge viter→vita."

Updated leverage ranking

Tier 1 — viter operational ownership (live, paying users):

1. **Own the Xero sync operations.** 15 tenant tokens, daily refresh, OAuth rotation, failure recovery. He wrote the equivalent in steve-agent. If `xero-sync` silently fails, Insperanto's reconciliation goes stale and Jeffrey notices. Mordechai is on the meeting extractor — nobody's watching.
2. **Statement extraction quality.** `extract-statement` is 601 LOC and v16 — the most-iterated function because PDFs break. 16 statements last 7d means real users are hitting edge cases. Yitzchak owning bug-triage here is direct Insperanto value.
3. **Diagnose `activity_log` death (last entry Apr 19).** Whatever stopped logging broke the `/me` dashboard's feed silently. Find why, fix, add a canary.

Tier 2 — bridge viter ↔ vita:

4. **Port `xero-sync` → vita adapter.** 387 LOC working template. New `xero_connection` source_type, raw Xero JSON → L0 → invoice/payment events → L1. Once vita has Xero data, the L2 synthesis question gets a real answer.
5. **Wire viter's `audit_log` (134K rows) as a vita L0 source type.** Massive structured corpus of "what happened in finance." Each row → L0 artifact + L1 event with typed actor/action/target. Trivial schema mapping; pure plumbing.
6. **Populate `mcp_tools_registry` from `viter-tools.ts`.** The 1,193-LOC tool file is the source of truth; the registry table is empty. A script + a few rows brings the `/me` "tools registry" tab to life.

Tier 3 — vita-side (still applies):

7. Gmail adapter (still missing in both)
8. First L2 synthesis run end-to-end
9. Fix `MIGRATIONS_FAILED` on vita main

Still Tier 4: dashboard UI

Final compressed principle

> *The dashboard wireframe was a distraction. There are three actual high-leverage zones:*
>
> 1. ***Operate viter*** *(live Xero pipeline, statement extraction, dead MCP feed) — paying-user value*
> 2. ***Bridge viter→vita*** *(port xero-sync, wire audit_log as L0, populate tool registry) — corpus expansion + reuse*
> 3. ***Run vita's first L2 synthesis*** *— unblocks every "where's L3" question downstream*
>
> *All three reuse what Yitzchak already knows (steve-agent OAuth + sync + Mongo). None of them are React.*

> *Branching Supabase isn't relevant to any of them.*